



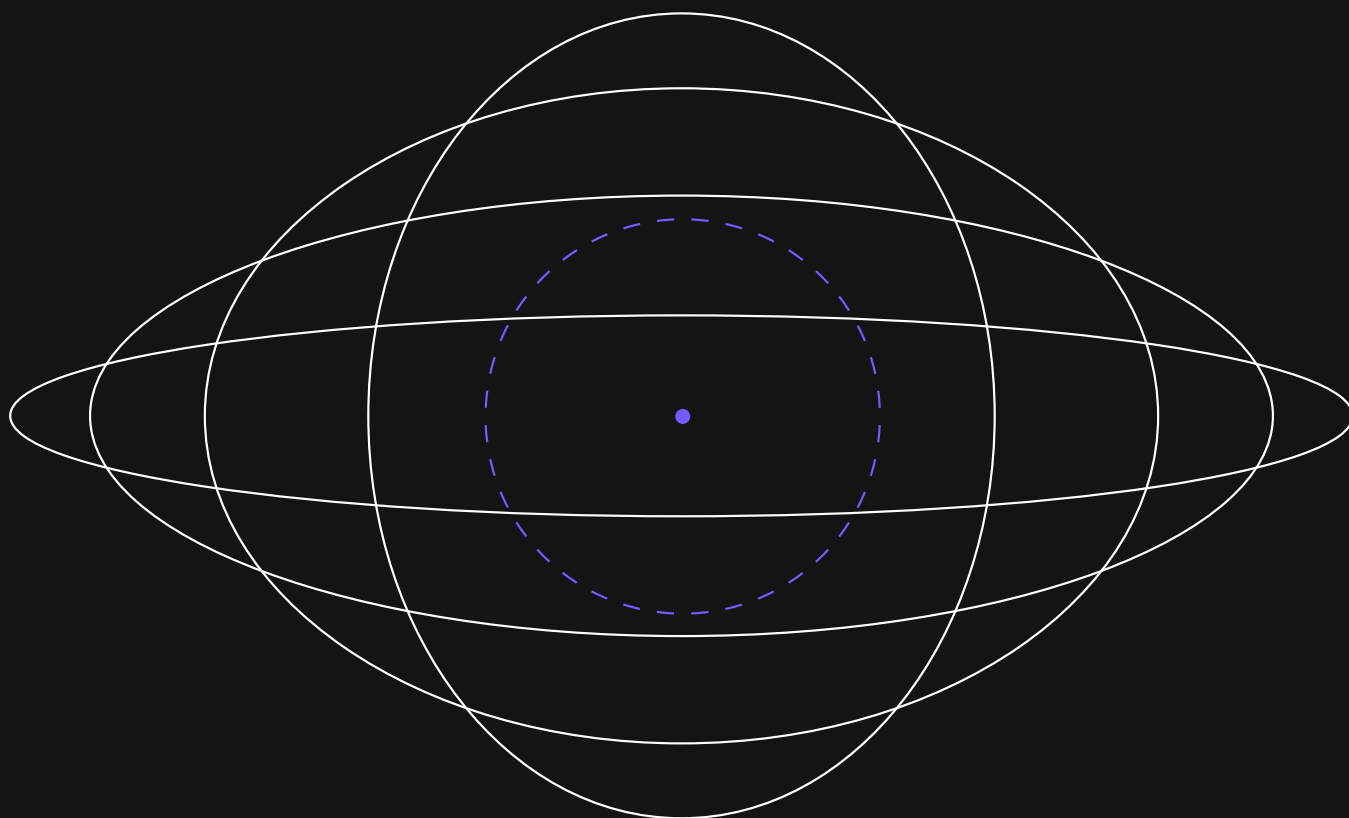
Машинное обучение

Лонгрид

Глубокое обучение

04

Задачи компьютерного зрения: детекция объектов



Содержание

1. Постановка задачи	3
2. Если бы объект был ровно один	3
3. Метрика качества: mAP	4
4. Как это делали до сетей: скользящее окно	6
5. R-CNN: регионы-кандидаты	6
6. Fast R-CNN: одна сеть на картинку	9
7. Faster R-CNN: сеть предлагает регионы	10
8. FPN: пирамида признаков	12
9. Одностадийные детекторы: YOLO	13
10. SSD	15
11. RetinaNet и focal loss	16
12. Итоги: три семейства детекторов	17
13. Что дальше	19

1. Постановка задачи

В этой теме очень много инженерии и заметно меньше математики, чем обычно, — поэтому лекция будет скорее обзорной: цель — общее представление о том, как задача детектирования объектов на изображении решается. При встрече с ней на практике детали конкретных алгоритмов добиваются из статей — имея общую картину, разобраться будет несложно.

Задача детекции — одна из классических в компьютерном зрении, и как только нейронные сети начали развиваться, это была одна из первых задач, к которой попытались подступиться сетевыми методами. Формулируется она очень просто: есть картинка, и на ней нужно обвести *ограничивающими прямоугольниками* (bounding boxes) все объекты, которые на ней находятся: сказать, что это за объект из предопределённого списка классов, и где он находится, — нарисовать рамку.

Пара уточнений к постановке. Детектируем мы обычно «вещи» (things) — людей, машины, знаки, то, что имеет форму и границы, — а не «материю» (stuff) вроде неба, дороги или травы: для последней есть сегментация из прошлой лекции. И рамка — не единственный вариант выхода: бывают повернутые рамки, эллипсы, попиксельные маски; но осевой прямоугольник — стандарт, с ним и работаем.

В плане разметки такая задача очень дешёвая — почти как классификация изображений: человек легко рисует рамки. Поэтому размеченных выборок существует много: и Pascal VOC, и MS COCO содержат соответствующую разметку (в COCO — около 200 тысяч изображений, 80 классов и полмиллиона объектов; поверх него есть и доразметка LVIS: больше тысячи классов, два миллиона масок); с обучающими данными проблем нет.

Сложности видны сразу из самой постановки. Первая: мы заранее **не знаем, сколько на картинке объектов** — один, два, десять или вообще ни одного. Вторая: объекты могут накладываться друг на друга, заслонять друг друга — и всё равно каждый нужно выделить. Более того, объекты бывают вложенными: если на картинке мотоцикл, а среди классов есть и «мотоцикл», и «колесо», то для одной и той же области изображения нужно выдать *несколько* прогнозов разного масштаба — рамку с мотоциклом и рамки с колёсами.

2. Если бы объект был ровно один

Полезно понять, почему задача с одним объектом проблемы не составляет. Если бы мы точно знали, что на каждой картинке ровно один объект, мы бы взяли обычную свёрточную сеть для классификации, в конце вывели бы, как всегда, вектор вероятностей классов, а рядом — четыре непрерывных числа: координаты левого верхнего угла, ширину и высоту рамки. Функция потерь состояла бы из двух слагаемых:

$$\mathcal{L} = \underbrace{\text{кросс-энтропия}}_{\text{метка класса}} + \lambda \cdot \underbrace{\sum (\text{выход сети} - \text{истинные параметры рамки})^2}_{\text{регрессия координат}},$$

где вместо суммы квадратов можно взять сумму модулей, а коэффициент λ настраивается как обычно. Классификация плюс регрессия — и задача решена. Все дальнейшие ухищрения нужны именно потому, что объектов *сколько-то* — заранее неизвестно сколько.

3. Метрика качества: mAP

Прежде чем переходить к методам, договоримся, что мерить, — по аналогии с точностью в классификации и средним Жаккардом в сегментации. На обучении мы, как всегда, минимизируем какую-то дифференцируемую функцию потерь; вопрос — как оценивать обученный детектор. Это нетривиально, потому что видов ошибок здесь несколько:

1. ошибка в **метке класса**;
2. ошибка в **положении**: объект найден, но рамка сдвинута;
3. **ложный прогноз**: рамка там, где никакого объекта нет;
4. **множественное срабатывание**: несколько близких рамок, наведённых одним и тем же объектом.

Все эти ошибки нужно собрать в один критерий. Стандартный — *mean Average Precision* (mAP), «дважды усреднённая точность». Разберём, как он устроен.

Локализация: IoU. Чтобы оценить, насколько хорошо предсказанная рамка совпала с истинной, используется уже знакомая по прошлой лекции мера Жаккара — *intersection over union*: площадь пересечения рамок делим на площадь объединения. Эта величина в алгоритмах детекции применяется буквально на каждом шагу.

Классификация: precision и recall. Зафиксируем класс — скажем, «знак стоп». Тогда

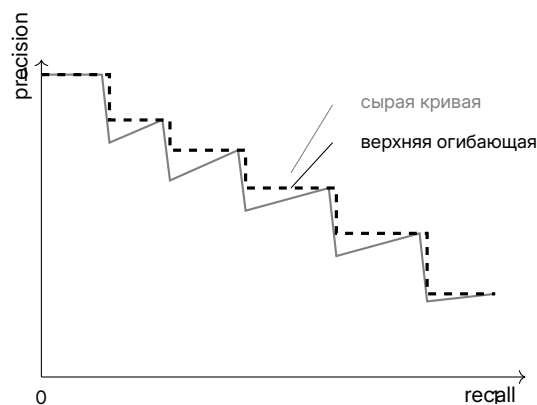
$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN} :$$

precision — из всех рамок, для которых мы предсказали «знак стоп», сколько действительно правильных; *recall* — из всех знаков стоп, которые есть на картинках, сколько мы нашли. Сюда упаковались все четыре вида ошибок: *false positive* — это не только рамка с IoU ниже порога или не на том объекте, но и *дублирующая* рамка на уже найденный объект (множественное срабатывание); *false negative* — объект, который не нашли вовсе *или* нашли, но уверенно ошиблись классом. У классификатора есть порог по вероятности класса: если порог задирать, *precision* растёт (остаются только самые уверенные прогнозы), а *recall* падает; если порог опускать — наоборот, *recall* стремится к единице, а *precision* может проваливаться. Чтобы получить величину, не зависящую от конкретного порога, строится *precision–recall-кривая*.

Как строится PR-кривая. Берём все предсказанные рамки данного класса и сортируем их по убыванию уверенности. Для каждой рамки решаем, правильная она или нет: если её IoU с какой-то истинной рамкой этого класса больше порога (например, 0,5) — это *true positive*, иначе *false positive*. Идя по отсортированному списку и пересчитывая *precision* и *recall* после каждой рамки, получаем кривую. Маленький пример (7 прогнозов, 5 истинных объектов):

Ранг	Правильно?	Precision	Recall
1	да	1,00	0,2
2	да	1,00	0,4
3	нет	0,67	0,4
4	нет	0,50	0,4
5	нет	0,40	0,4
6	да	0,50	0,6
7	да	0,57	0,8

Видно и падение precision на ложных срабатываниях, и его подрастание на шестом-седьмом прогнозах: по кривой precision меняется немонотонно. Поэтому на практике берут её *верхнюю огибающую*: в каждой точке recall ставится наибольший precision, достигавшийся при этом recall или правее.



AP и усреднения. Average precision — площадь под огибающей; на практике её считают по сетке значений recall 0, 0,1, ..., 1, суммируя «столбики». Это было первое усреднение — по всем возможным порогам классификатора. Второе усреднение — по классам:

$$mAP = \frac{1}{K} \sum_{k=1}^K AP_k.$$

Иногда делается и третье: порог на IoU, отделяющий «правильную» локализацию (те самые 0,5), — тоже произвольный выбор, поэтому усредняют и по сетке порогов IoU, начиная с 0,5 и повышая. Важно одно: у каждой стандартной выборки зафиксирован свой протокол вычисления, чтобы разные детекторы можно было сравнивать между собой. Чем mAP выше, тем лучше.

4. Как это делали до сетей: скользящее окно

Прежде чем идти к нейросетевым методам, полезно увидеть досетевую схему — многое из неё перекочевало дальше. Детекция сводилась к классификации перебором: берём окно фиксированного размера, двигаем его по всем позициям изображения и в каждой позиции спрашиваем классификатор: «есть здесь объект нужного класса?» Чтобы ловить объекты разных размеров — повторяем на нескольких масштабах (пирамида уменьшенных копий изображения) и с несколькими соотношениями сторон окна.



Одно и то же окно на пирамиде уменьшенных копий изображения ловит объекты разных размеров.

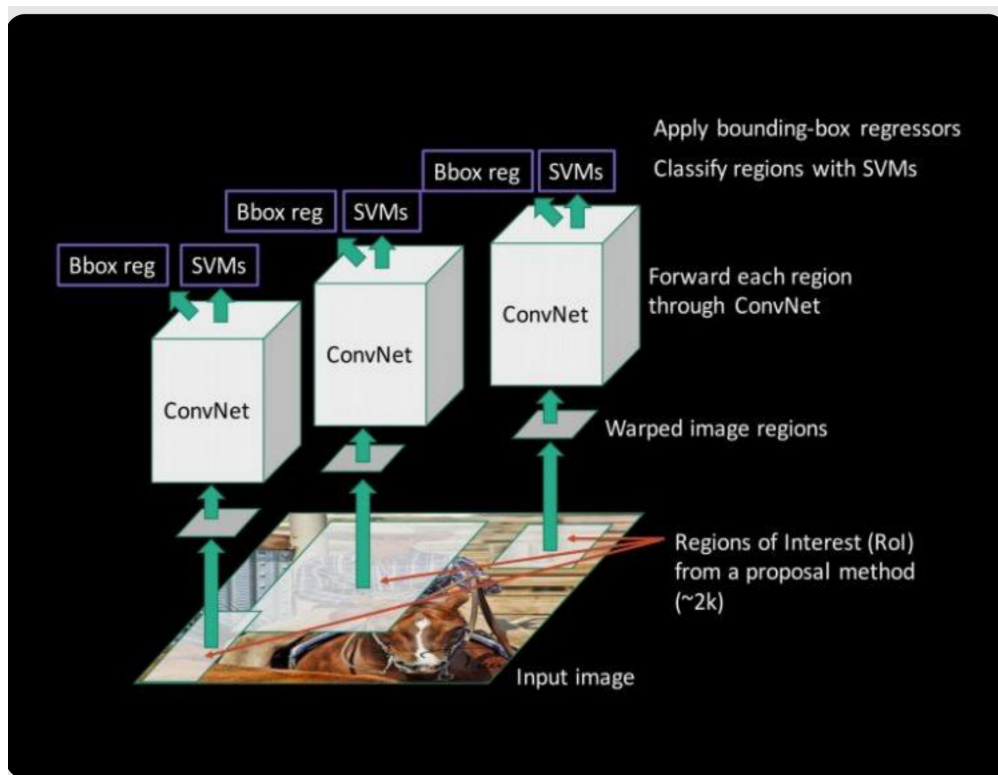
Уже здесь появляются два сюжета, которые пройдут через всю лекцию.

Первый — *подавление немаксимумов*: раз окна перебираются густо, вокруг каждого объекта срабатывает целое семейство окон, и нужно оставить по одному — эту процедуру мы подробно разберём ниже.

Второй — *дисбаланс классов*: окон с объектом — крохи, окон с фоном — миллионы. Классический приём называется *hard negative mining*: сначала учим классификатор на случайных фоновых окнах; затем в цикле — прогоняем детектор по обучающим картинкам, собираем *трудные* ложные срабатывания (фон, который детектор перепутал с объектом), добавляем их в обучающую выборку и переучиваем. Классификатор концентрируется на сложных фоновых примерах — этот приём стоит запомнить: в конце лекции встретится его прямой наследник, устроенного уже не итерациями, а одной формулой.

5. R-CNN: регионы-кандидаты

Первый нейросетевой подход — R-CNN (regions with CNN features, 2013 год). Логика такая: классифицировать картинку мы умеем, но не знаем, сколько на ней объектов и где они. Будем действовать в два шага: сначала каким-то алгоритмом выделим на картинке множество *регионов-кандидатов* (regions of interest) — прямоугольников, в которых можно ожидать объекты. Пусть их будет много, пусть будут лишние — не страшно. А затем каждый регион по отдельности, как самостоятельную картинку, пропустим через свёрточную сеть и классифицируем.



Конвейер R-CNN: картинка → ~2000 регионов-кандидатов → warp → CNN → классификация каждого региона (Girshick et al., 2013).

Генерация кандидатов: selective search. В R-CNN регионы-кандидаты порождались алгоритмом selective search — сложной инженерной процедурой ещё досетевой эпохи. Детально понимать её устройство не обязательно (через пару шагов мы вообще от неё откажемся), но на верхнем уровне это *иерархическая кластеризация*: картинка разбивается на суперпиксели — вот и они пригодились, — которые затем постепенно сливаются во всё большие регионы по рукотворным признакам: цветовым, текстурным (гистограммы градиентов — классика досетевого зрения), позиционным. Плюс отсев почти дублирующих прямоугольников. На выходе — порядка двух тысяч регионов-кандидатов на картинку.



Figure 2: Two examples of our selective search showing the necessity of different scales. On the left we find many objects at different scales. On the right we necessarily find the objects at different scales as the girl is contained by the tv.

Selective search: суперпиксели сливаются во всё более крупные регионы; нужны разные масштабы — слева объекты находятся на мелких, справа девушка «в телевизоре» — только на крупном (Uijlings et al., 2013).

Классификация региона. Каждый регион-кандидат имеет свою форму — вытянутую, квадратную, какую угодно, — а свёрточная сеть, хоть формально и может принимать вход любого размера,

на практике работает с фиксированным. Поэтому каждый регион сначала приводится (warp) к стандартному размеру вроде $224 \times 224 \times 3$, а затем пропускается через свёрточную сеть — как правило, предобученную на ImageNet. Показательная деталь эпохи: 2013 год, нейросетям ещё не доверяли решать задачу целиком — CNN здесь работает только *экстрактором признаков*, а решения поверх неё принимают классические модели. Обучались две вещи:

- **классификаторы** — причём отдельный бинарный классификатор (SVM) на каждый класс: «это знак стоп или нет», «это машина или нет»;
- **корректирующие регрессоры**: выделенный алгоритмом прямоугольник наверняка неточно совпадает с истинным, поэтому сеть учится предсказывать четыре поправки — как сдвинуть угол и подправить ширину и высоту, чтобы лучше попасть в разметку.

Про поправки стоит сказать точнее — параметризация тут не наивная. Если $(\hat{x}, \hat{y}, \hat{w}, \hat{h})$ — регион-кандидат, а (x, y, w, h) — истинная рамка, то целевые значения регрессии —

$$\left(\frac{x - \hat{x}}{\hat{w}}, \frac{y - \hat{y}}{\hat{h}}, \ln \frac{w}{\hat{w}}, \ln \frac{h}{\hat{h}} \right) :$$

сдвиги измеряются в долях размера региона, а изменения размеров — логарифмами отношений. Все четыре величины безразмерны, поэтому один и тот же регрессор одинаково хорошо правит и большие, и маленькие рамки; обратный пересчёт в координаты очевиден. Хороший вопрос для самопроверки: а почему именно *логарифмы* отношений размеров? Потому что ошибка «на один пиксель» для огромной рамки и для крошечной — совсем разные ошибки: важен порядок изменения размера, а не его абсолютная величина; в логарифмической шкале «увеличить вдвое» и «уменьшить вдвое» — симметричные шаги $\pm \ln 2$, и квадратичная функция потерь штрафует их одинаково. Эта параметризация дальше используется во всех детекторах лекции.

Разметка регионов для обучения. Сгенерированный регион не совпадает точно ни с одной истинной рамкой, поэтому метки назначаются через IoU — причём, любопытно, на разных стадиях по-разному (в статье этому посвящён отдельный аппендикс). При дообучении свёрточной сети позитив — регион с $\text{IoU} \geq 0,5$ с какой-то истинной рамкой, остальное — фон. А вот при обучении SVM позитивы — вообще не регионы, а *сами истинные рамки*; фоном объявляется регион, пересекающийся со всеми истинными рамками меньше чем на 0,3, а «серая зона» между ними в обучение просто не попадает. Авторы нашли эти пороги перебором: попытка учить SVM с порогом 0,5 роняла mAP на пять пунктов. Наконец, регрессоры поправок учатся только на парах «регион–истина» с $\text{IoU} \geq 0,6$ — править имеет смысл только рамку, которая уже почти попала.

Обучение в три стадии. Сначала предобученная свёрточная сеть дообучается через softmax на номенклатуру классов задачи (плюс фон) — голова от ImageNet, понятно, выбрасывается. Дисбаланс «объект/фон» здесь лечится ресемплингом батча: в оригинале — 32 окна с объектами плюс 96 фоновых из двух тысяч регионов картинки. Затем на признаках сети обучаются отдельные SVM для каждого класса. Наконец, дообучаются регрессоры поправок. Три последовательные, по сути независимые процедуры.

Тест и подавление немаксимумов. На тестовой картинке: генерируем регионы selective search'ем, каждый приводим к стандартному размеру, пропускаем через сеть, получаем вероятности классов и поправки. Затем — *жадное подавление немаксимумов* (non-maximum suppression, NMS): близкие регионы наверняка отнесены к одному классу, и нужно оставить по одной рамке на объект. Для каждого класса: фоновые регионы выбрасываем; в цикле по оставшимся — если два прогнозных региона одного класса пересекаются сильнее порога (по IoU, 0,5–0,7), оставляем тот, у которого

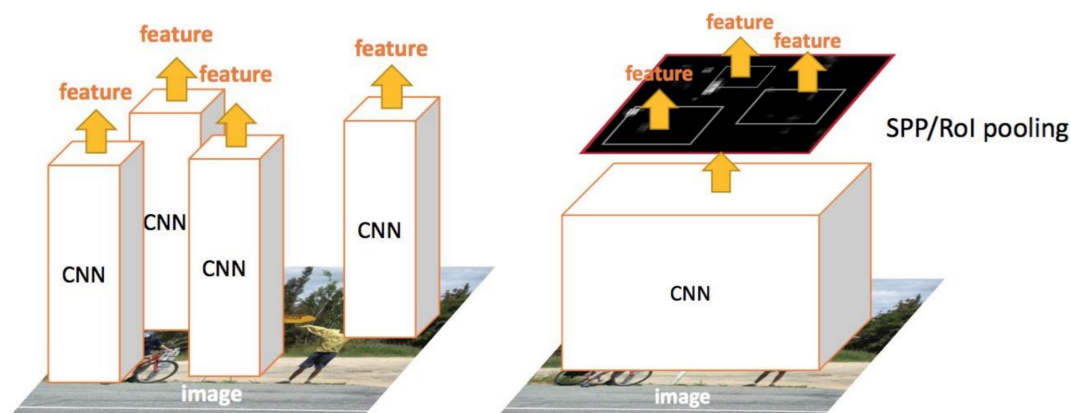
вероятность класса выше. Жадное — потому что результат, вообще говоря, зависит от порядка обхода, но главное достигается: двух почти совпадающих рамок одного класса на выходе не будет. Если один регион оказался отнесён к двум классам сразу (для независимых классификаторов такое не запрещено) — берём класс с наибольшей уверенностью.

Главный недостаток — скорость. Обучение долгое (три стадии), но хуже другое: на *тесте*, где детектору положено быть быстрым — машина едет, объекты нужно распознавать на лету, — каждая картинка порождает тысячи регионов, и все они прогоняются через сеть по отдельности. Фактически батч размера в тысячи на одну картинку. Качество приличное, скорость — нет.

6. Fast R-CNN: одна сеть на картинку

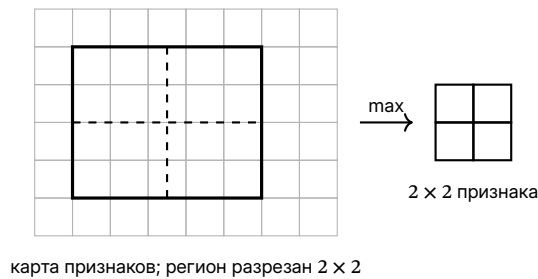
Следующая итерация от тех же авторов — Fast R-CNN (2015). Ключевая идея: свёрточная сеть применяется к картинке **один раз**. Берём входное изображение целиком, приводим к стандартному размеру, пропускаем через свёрточную сеть до какого-то промежуточного слоя — и всю работу с регионами ведём уже на этой *карте признаков*, а не на исходной картинке.

Регионы-кандидаты пока генерирует всё тот же selective search — на исходном изображении. Но теперь мы берём их *относительные* координаты (от 0 до 1) и по ним вырезаем соответствующий блок из карты признаков — просто пропорционально, без пересчёта каких-либо зависимостей.



Слева — R-CNN: своя свёрточная сеть на каждый регион; справа — Fast R-CNN: одна сеть на картинку, регионы вырезаются уже из карты признаков (SPP/RoI pooling).

RoI pooling. Вырезанные блоки имеют разный размер — регионы-то разные, — а классификатору нужен фиксированный. Здесь заимствуется идея пирамидального пулинга из прошлой лекции: как там карта резалась на блоки 2×2 , 3×3 с пулингом в каждом блоке, так и здесь вырезанный кусок карты признаков режется на фиксированную сетку (скажем, 2×2), и в каждой ячейке берётся максимум по каждому каналу. Какого бы размера ни был регион, на выходе всегда одинаковое число признаков. Это и есть RoI pooling. К слову, программировать его неприятно: это не свёрточный слой — для каждого региона своя маска, своё разбиение, и чтобы всё работало быстро, авторам пришлось спускаться до CUDA-ядер. (Забегая вперёд: позже появилась аккуратная версия этой операции — RoI align, где границы ячеек не округляются до целых пикселей, а значения берутся билинейной интерполяцией; именно она используется в Mask R-CNN.)

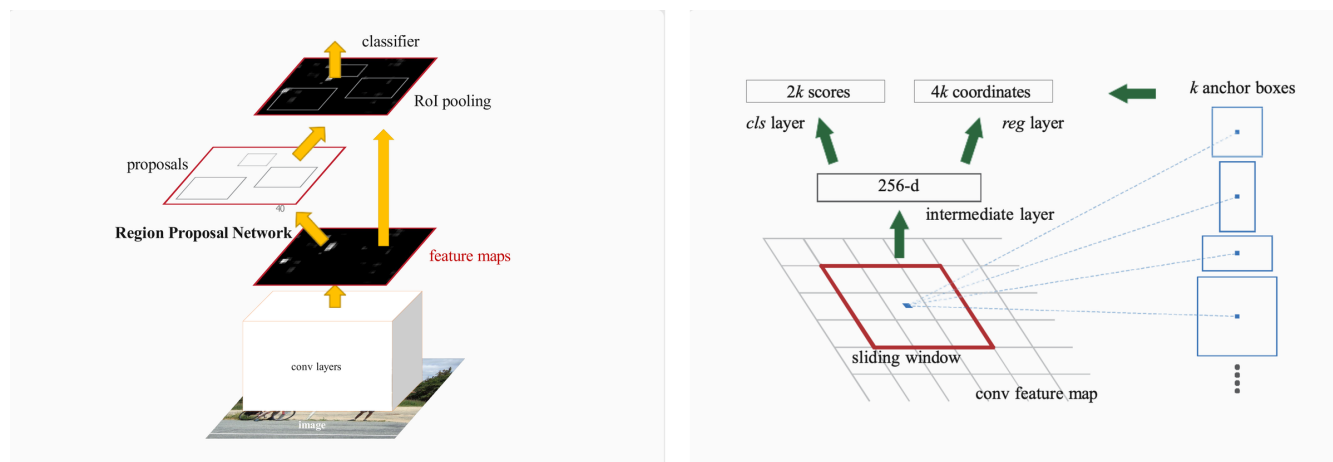


Единая голова и единое обучение. Вместо трёх независимых стадий R-CNN здесь одно сквозное обучение: над RoI pooling стоят полносвязные слои, из которых выводятся, во-первых, один softmax сразу на все классы *плюс класс «нет объекта»* (фон всегда добавляется), и во-вторых, регрессоры поправок — свои для каждого класса, всего $4K$ выходов: мотоцикл большой — рамку нужно править одним образом, колесо маленькое — другим. На тесте сначала смотрим softmax: если победил фон — регион выбрасываем, иначе применяем поправки соответствующего классу регрессора. В конце, как всегда, — подавление немаксимумов — эта операция в том или ином виде есть в конце любого детектора.

7. Faster R-CNN: сеть предлагает регионы

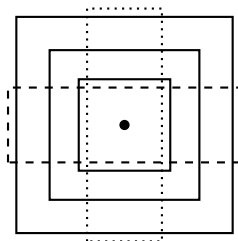
Следующий шаг той же линейки — Faster R-CNN (тот же 2015 год). Авторы заметили: selective search — чужеродная деталь. Она унаследована из досетевой эпохи, работает на рукотворных признаках и — главное — медленно: выделить тысячи регионов на тестовой картинке занимает заметное время. Раз уж мы учим нейросеть понимать изображение, пусть и регионы-кандидаты предлагает такая же сеть. Так появляется *Region Proposal Network* (RPN): входное изображение проходит через свёрточную сеть, и по карте признаков отдельная маленькая свёрточная сеть генерирует набор регионов, где возможны объекты; дальше всё — в точности как в Fast R-CNN (RoI pooling, классификатор, регрессоры).

Скользящее окно и якоря. Как RPN ищет регионы? Полным перебором: окно 3×3 ставится в каждую позицию карты признаков. Из признаков окна ($3 \times 3 \times$ число каналов) выводится небольшое представление (в оригинале — 256 признаков), а из него — прогнозы. Технически никакого цикла по позициям нет: «окно в каждой позиции» — это просто свёртка 3×3 , за которой идёт свёртка 1×1 до 256 каналов.



Faster R-CNN: слева — RPN поверх карты признаков предлагает регионы, дальше RoI pooling и классификатор; справа — скользящее окно 3×3 , промежуточные 256 признаков и выходы $2k$ оценок и $4k$ координат для k якорей (Ren et al., 2015).

Чтобы регионы могли быть разного размера и формы, вводятся *якоря* (anchors): в каждой позиции рассматривается фиксированный набор стандартных рамок с центром в этой точке — в оригинале три масштаба (128^2 , 256^2 , 512^2 в координатах исходного изображения) на три соотношения сторон ($1:1$, $1:2$, $2:1$), итого $k = 9$ якорей на позицию. Для каждого якоря сеть выводит $2k$ оценок — «есть объект / нет объекта» (маленький softmax из двух) — и $4k$ чисел — корректирующие поправки рамки.



якоря: 3 масштаба $\times$ 3 соотношения сторон, центр — текущая позиция

Разметка якорей. Метки назначаются опять через IoU: если якорь (после поправок) пересекается с какой-то истинной рамкой сильнее порога 0,7 — класс «объект есть». А чтобы ни одна истинная рамка не осталась без якоря (бывает, что ни один якорь не дотянул до 0,7), для каждой истинной рамки к положительным добавляется якорь с наибольшим IoU.

Отбор кандидатов. Якорей на карте признаков — девять на каждую позицию, очень много. Поэтому на обучении: якоря, вылезавшие за границу изображения, отбрасываются (на тесте их не отбрасывают, а обрезают по границе); оставшиеся сортируются по вероятности объекта, берётся топ-6000; к ним применяется подавление немаксимумов; после него оставляют лучшие 2000 на обучении (на тесте — порядка 300). Это и есть регионы-кандидаты, которые идут дальше в Fast R-CNN-часть.

Совместное обучение. Две конструкции — RPN и детектирующая голова — скрещивались четырёхстадийной процедурой: (1) берём ImageNet-предобученную сеть, учим RPN (свёрточные слои дообучаются); (2) фиксируем RPN как генератор регионов, учим Fast R-CNN; (3) снова дообучаем RPN под изменившиеся свёрточные слои; (4) объединяем всё в одну модель и дообучаем все параметры вместе, end-to-end.

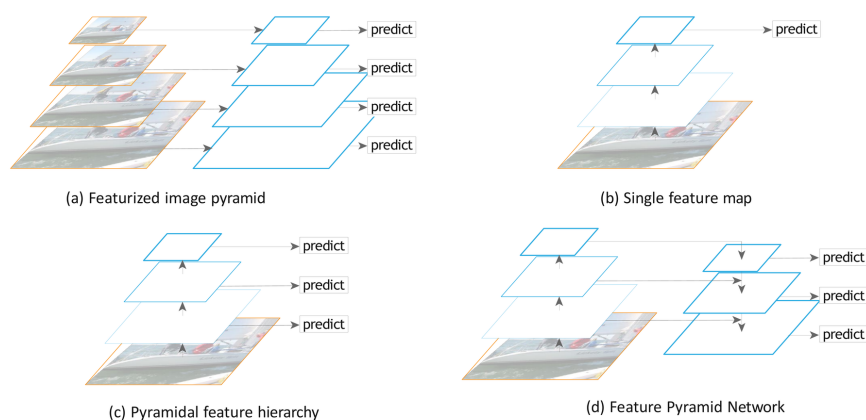
Тест. Картинка → свёрточная сеть → RPN (это две маленькие свёртки — несравнимо быстрее selective search) → RoI pooling → классификатор и регрессоры → подавление немаксимумов. Методы этого класса называются *двухстадийными* детекторами: сначала генерируется разреженный набор регионов-кандидатов, потом по каждому решается задача классификации и уточнения. Faster R-CNN до сих пор остаётся классическим детектором, который много где используется. По скорости вся эта эволюция дала выигрыш на два порядка (время на одну тестовую картинку):

Метод	Время на кадр
R-CNN	49 с
SPP-Net	4,3 с
Fast R-CNN	2,3 с
Faster R-CNN	0,2 с

(SPP-Net — промежуточная модель между R-CNN и Fast R-CNN: именно она первой предложила прогонять сеть по картинке один раз и вырезать регионы из карты признаков пирамидальным пулингом.) Выигрыш не только на тесте: обучение R-CNN занимало 84 часа, Fast R-CNN — 8,75; и при этом качество даже подросло (mAP 66,0 → 68,1 на Pascal VOC) — сквозное обучение одной головы оказалось лучше трёх отдельных стадий. Но даже 0,2 секунды — это ~ 5–7 кадров в секунду: до реального времени (25 кадров) всё ещё не дотягивает.

8. FPN: пирамида признаков

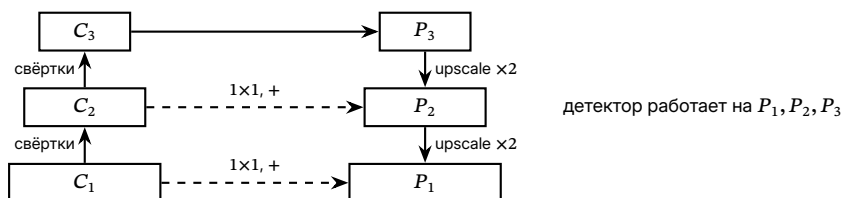
Ещё одно улучшение, которое обычно добавляют при реализации Faster R-CNN, — *Feature Pyramid Network* (FPN). Это вторая идея, заимствованная у сегментаторов (первой был пирамидальный пулинг, породивший RoI pooling). В сегментации выяснилось, что работать от одного сильно сжатого представления невыгодно — нужен декодер со skip connections на соответствующих разрешениях. Здесь то же самое: до сих пор все якоря и прогнозы строились по одной сжатой карте признаков.



Четыре способа работать с масштабами: (a) пирамида копий изображения — дорого; (b) одна сжатая карта — грубо; (c) прогнозы с промежуточных слоёв — ранние карты слабые; (d) FPN: проход назад с латеральными

связями (Lin et al., 2017).

FPN добавляет к обычному проходу вперёд простой проход назад: очередное огрублённое представление увеличивается вдвое (upscale), к нему по горизонтальной связи подмешивается представление соответствующего масштаба из прохода вперёд — свёртка 1×1 , чтобы согласовать число каналов, и простое сложение, как в LinkNet, — а результат подрабатывается свёрткой 3×3 . Получается несколько уровней признаковых представлений разных масштабов.



Якоря теперь распределяются по уровням: раз масштабы уже разведены по пирамиде, на каждом уровне достаточно якорей *одного* масштаба с тремя соотношениями сторон — один и тот же элемент 3×3 на мелкой карте соответствует большому куску исходного изображения, на крупной — маленькому, так что разные размеры получаются автоматически. Тонкий момент: когда регион уже найден и нужно вырезать для него признаки, с какого уровня пирамиды их брать? Стандартное правило — по размеру региона:

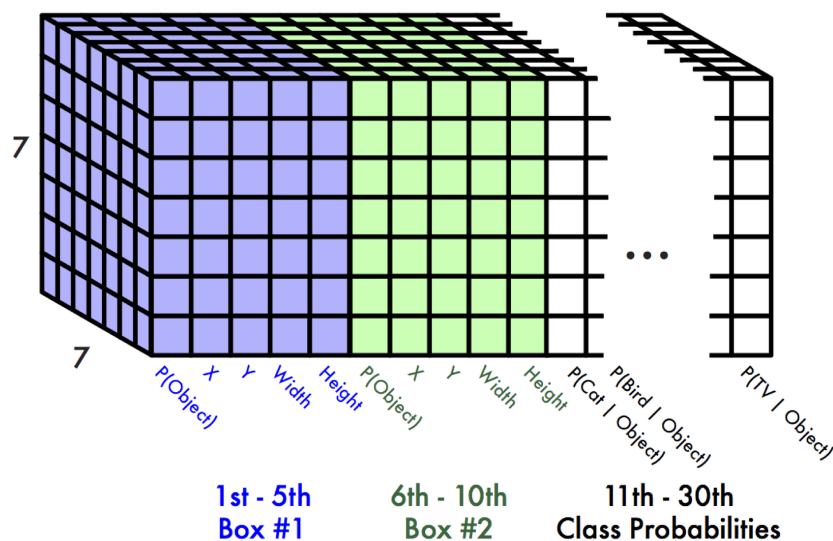
$$k = \left\lfloor k_0 + \log_2(\sqrt{wh} / 224) \right\rfloor :$$

регион размером с «канонические» 224×224 идёт на базовый уровень k_0 , вдвое больший — на уровень грубее, вдвое меньший — на уровень детальнее. Дальше вся процедура — как в Faster R-CNN.

9. Одностадийные детекторы: YOLO

Всё, что было выше, — двухстадийные детекторы: сначала регионы-кандидаты, потом детекция по ним. Возникла естественная идея: а нельзя ли одним проходом вперёд получить сразу всё — и регионы, и метки? Оказывается, можно.

Самое знаменитое имя здесь — YOLO, «you only look once» (2015): один взгляд на картинку — и готовы все детекции. Устроено так. Картинка режется регулярной сеткой на $S \times S$ ячеек (в оригинале 7×7). Каждая ячейка предсказывает B рамок (в оригинале две) — каждая с координатами и уверенностью $P(\text{Object})$, что в рамке вообще есть объект, — и один общий вектор вероятностей классов. Весь выход сети — один тензор размера $S \times S \times (5B + K)$, никаких отдельных стадий.



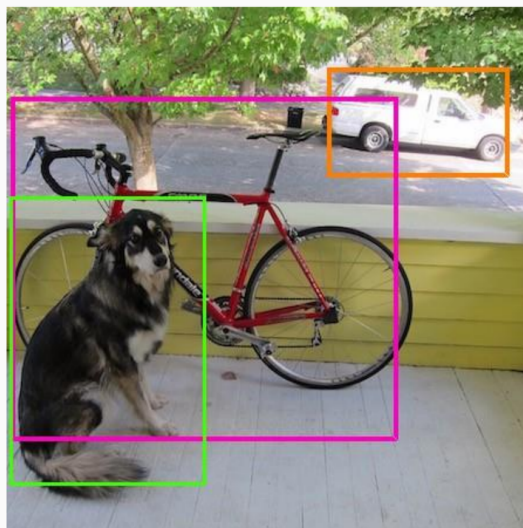
Выход YOLO: сетка 7×7 , по 5 чисел на каждую из двух рамок ($P(\text{Object})$, x , y , w , h) и 20 вероятностей классов.

Обучение: каждый истинный объект приписывается той ячейке, куда попал его *центр*, а внутри ячейки — той из B рамок, которая ближе к нему по форме; её $P(\text{Object})$ подтягивается вверх, у остальных рамок ячейки — вниз; ячейки, которым не достался ни один объект, учат все свои рамки выдавать низкую уверенность. Забавная деталь: функция потерь — квадратичная ошибка *для всего сразу*, и для координат, и для уверенностей, и даже для вероятностей классов.

Combine bboxes and class probabilities



Apply NMS and probability thresholding



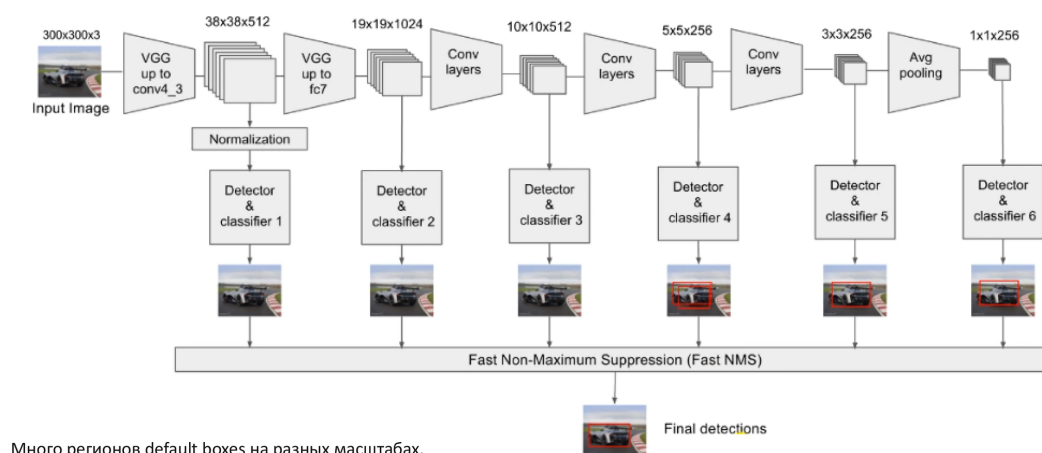
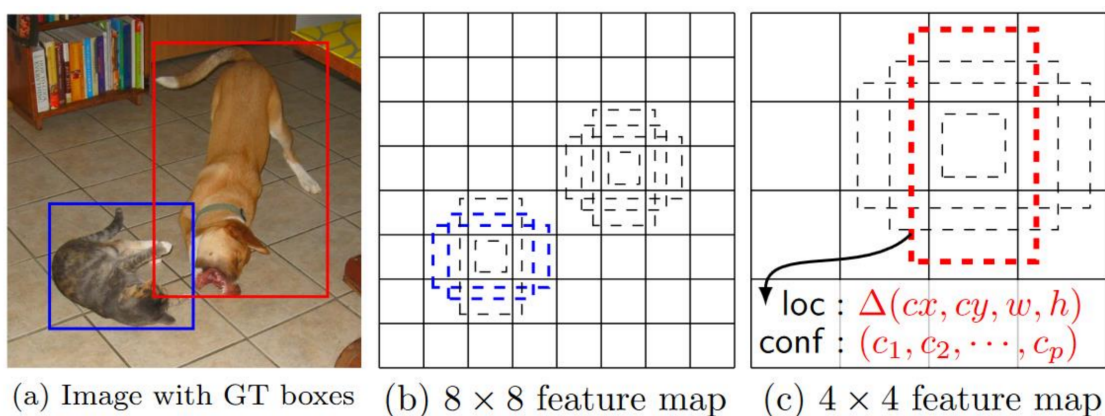
Слева — все рамки всех ячеек с их вероятностями классов; справа — после порога по уверенности и подавления немаксимумов: собака, велосипед, машина.

Инженерия: первые 20 свёрточных слоёв предобучены на ImageNet; вход увеличен до 448×448 (сеть дообучали на большем разрешении, чем классификационные 224); всюду Leaky ReLU, bottleneck-свёртки 1×1 , dropout, аугментации. Результат — честный real-time: 45 кадров в секунду, а облегчённая YOLO-tiny — 155. Цена: в каждой ячейке — максимум B объектов, поэтому толпа и мелкие объекты, чьи центры сыплются в одну ячейку, детектируются плохо; и по mAP YOLO уступала двухстадийным.

10. SSD

Следующий одностадийный детектор — SSD, single shot detector. Работает он просто. Идёт обычная свёрточная сеть; на промежуточном представлении какого-то масштаба — скажем, сетке 8×8 — в каждую позицию подставляются якоря (например, три: квадратный, вытянутый по горизонтали, вытянутый по вертикали). Каждый якорь соответствует какому-то прямоугольнику исходного изображения; те якоря, что пересекаются с истинной разметкой сильнее порога, получают метку соответствующего класса — скажем, «кошка», — остальные считаются фоном. Для каждого якоря сеть выводит вектор вероятностей за все классы плюс четыре поправки рамки; вывод делается свёртками 3×3 прямо над соответствующим представлением.

И так — не для одного пространственного масштаба, а для нескольких: по мере того как сеть сужается, те же свёртки-детекторы ставятся над несколькими уровнями, чтобы ловить объекты разных размеров. Всё обучается одним проходом: и «где регионы», и «какие в них классы» — одновременно. В конце — обязательное подавление немаксимумов: один объект может быть задетектирован на разных масштабах.

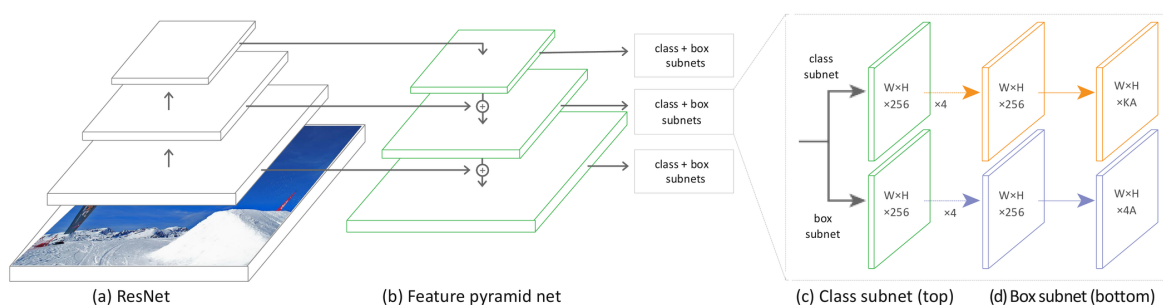


SSD: сверху — якоря на сетках 8×8 и 4×4 и их привязка к разметке (кошка — синие, собака — красные); внизу — архитектура: backbone VGG-16, дополнительные сужающиеся слои и детектор с классификатором над каждым масштабом, в конце — общий NMS (Liu et al., 2016).

В оригинальной конфигурации SSD (2016) выдаёт 8732 якоря-детекции на картинку и работает со скоростью около 59 кадров в секунду при mAP 74,3 на Pascal VOC. От YOLO он отличается по нескольким пунктам: отдельной уверенности $P(\text{Object})$ нет — сразу softmax по классам с фоном; backbone — стандартная VGG (у YOLO была своя сеть Darknet); дисбаланс «объект/фон» лечится hard negative mining — тем самым приёмом из досетевой эпохи, с соотношением положительных к отрицательным 1:3; а главное, якоря на нескольких масштабах снимают ограничение «В объектов на ячейку» — толпа перестаёт быть проблемой.

11. RetinaNet и focal loss

Дальше идеи складываются одна с другой: RetinaNet (2017) — тоже одностадийный детектор, но вместо сужающейся сети — та самая пирамида FPN поверх ResNet, и над каждым уровнем пирамиды ставятся две маленькие свёрточные подсети: class subnet (классы для каждого якоря) и box subnet (поправки рамок). Плюс одна важная добавка — *focal loss*.



RetinaNet: ResNet-backbone, пирамида признаков FPN и две подсети — классификационная и регрессионная — на каждом уровне (Lin et al., 2017).

Наблюдение такое: среди всех якорей-кандидатов положительных (объект) — крохи, а отрицательных (фон) — на порядки больше. Ресемплинг внутри минибатча — взять поровну положительных и отрицательных — помогает, но принципиально дисбаланс никуда не девается. Разберём стандартное кросс-энтропийное слагаемое — $-\log p$, где p — вероятность правильного класса. Лёгкие примеры — очевидный фон, где p близко к единице, — дают каждый понемногу, но их так много, что суммарно они перевешивают вклад немногочисленных трудных примеров. Focal loss домножает кросс-энтропию на затухающий коэффициент:

$$\mathcal{L}_{\text{focal}} = -(1 - p)^{\gamma} \log p.$$

При p , близком к единице, множитель $(1 - p)^{\gamma}$ практически обнуляет вклад — лёгкие примеры перестают участвовать; в области малых p — там, где примеры трудные, — функция почти не отличается от обычной кросс-энтропии. Обучение концентрируется на сложном.



Параметр γ управляет «закрученностью»; лучше всего у авторов сработало $\gamma = 2$ — прирост качества существенный. RetinaNet с FPN и focal loss — одностадийный детектор уровня state-of-the-art своего времени, который много где используется.

12. Итоги: три семейства детекторов

Сведём качество и скорость в одну таблицу (Pascal VOC 2007):

Метод	mAP	Кадров/с
DPM v5 (досетевой)	33,7	0,07
R-CNN	66,0	0,05
Fast R-CNN	70,0	0,5
Faster R-CNN	73,2	7
YOLO	69,0	45

Вся история лекции — в пяти строках: нейросети сразу удвоили качество досетевых методов, двухстадийная линейка наращивала и качество, и скорость, а одностадийная YOLO разменяла несколько пунктов mAP на порядок скорости.

Если посмотреть на всю картину, детекторы делятся на три группы.

Двухстадийные (R-CNN → Fast R-CNN → Faster R-CNN): сначала разреженный набор регионов-кандидатов, потом детекция по ним. Качество — лучшее: отдельная сеть, предлагающая регионы, качество повышает. Но даже Faster R-CNN — это порядка 7 кадров в секунду: для реального времени недостаточно.

Одностадийные (YOLO, SSD, RetinaNet): один проход вперёд, всё сразу. Быстрее, качество чуть проседает относительно двухстадийных — таков размен.

Вот как выглядит карта методов, если расписать группы по именам: двухстадийные — R-CNN, Fast R-CNN, Faster R-CNN (обычно с FPN), Mask R-CNN (знакомая по прошлой лекции instance-сегментация — тоже член этого семейства), Cascade R-CNN, Hybrid Task Cascade; одностадийные якорные — YOLO, SSD, RetinaNet; одностадийные безъякорные — FCOS, CornerNet, CenterNet, CentripetalNet (о них — ниже).

Anchor-free — современное поколение. Якоря тянут за собой целый хвост инженерных выборов: какие масштабы, какие соотношения сторон, как устраивать сопоставление (matching) якорей с разметкой. В anchor-free-методах классифицируются не заякоренные регионы, а отдельные точки — и весь этот хвост отпадает.

Чтобы слово не осталось пустым, — один представитель, FCOS (fully convolutional one-stage). Каждая точка изображения, попавшая внутрь истинной рамки, считается положительной и предсказывает не (x, y, w, h) , а четыре расстояния до сторон своей рамки — (l, t, r, b) (влево, вверх, вправо, вниз), плюс вектор классов, плюс отдельное число — *center-ness*, оценку того, насколько точка близка к центру объекта (учится обычной бинарной кросс-энтропией). На тесте *center-ness* домножается на вероятность класса — и кривые рамки, предсказанные точками с краёв объекта, выпадают из отбора ещё до подавления немаксимумов.

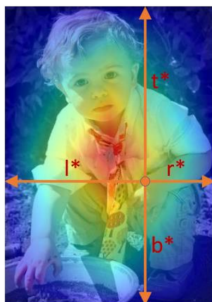


Figure 3 – Center-ness. Red, blue, and other colors denote 1, 0 and the values between them, respectively. Center-ness is computed by Eq. (3) and decays from 1 to 0 as the location deviates from the center of the object. When testing, the center-ness predicted by the network is multiplied with the classification score thus can down-weight the low-quality bounding boxes predicted by a location far from the center of an object.

В каждой точке предсказывается (l, t, r, b) , а не (x, y, w, h) + класс (метка из C) + центральность (число).

$$\text{centerness}^* = \sqrt{\frac{\min(l^*, r^*)}{\max(l^*, r^*)} \times \frac{\min(t^*, b^*)}{\max(t^*, b^*)}}.$$

Center-ness на $[0, 1]$ обучается с помощью binary cross-entropy (BCE) loss.

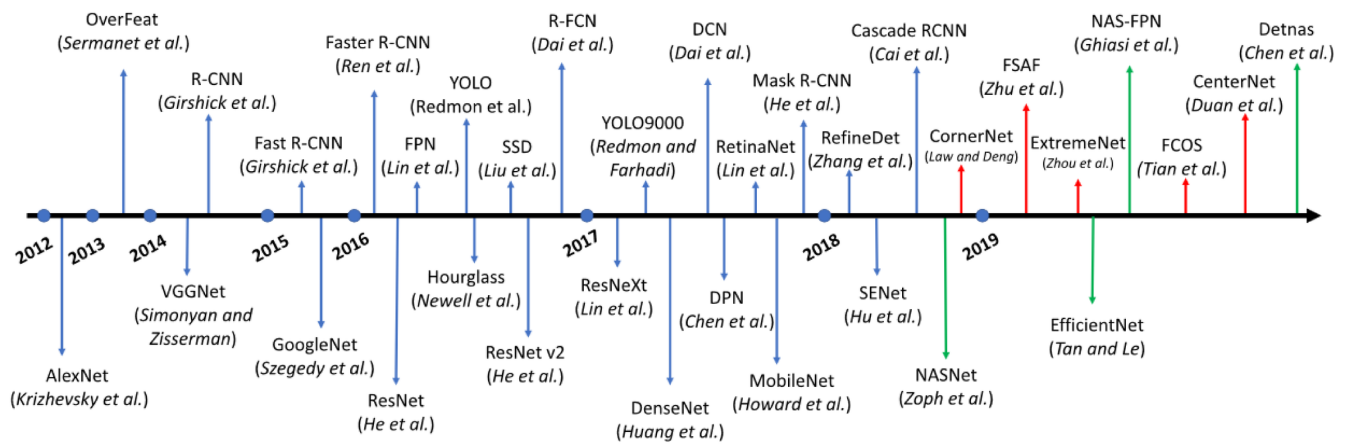
При работе к точкам с большим значением можно применить NMS.

В классификации вместо softmax — мультикласс.

FCOS: точка предсказывает расстояния (l, t, r, b) до сторон рамки; $\text{center-ness} = \sqrt{\frac{\min(l, r)}{\max(l, r)} \cdot \frac{\min(t, b)}{\max(t, b)}}$ — максимален в центре объекта и затухает к краям (Tian et al., 2019).

Работает поверх FPN (уровни P_3 – P_7 ; слишком большие для данного уровня рамки просто не детектируются на нём), обучается focal loss'ом — и обходит по качеству якорные аналоги. Прочие имена этой группы — CornerNet, CenterNet, CentripetalNet; а детекция трансформером — DETR, где рамки выдаёт декодер напрямую, без якорей и NMS, — подождёт блока про механизм внимания.

Методы этой группы выходят каждый год; при решении задачи детекции на практике, когда важно лучшее качество, стоит открыть свежую статью и посмотреть, как там устроена процедура: с общим представлением из этой лекции разобраться будет несложно.



История детекции с 2012 года: сверху — детекторы (красным — anchor-free), снизу — backbone-сети, на которых они строились.

13. Что дальше

На этом завершаем — и лекцию, и весь «картинный» блок курса: полносвязные сети, свёрточные сети и их применения к разным задачам компьютерного зрения. Дальше курс выходит за пределы решётки: на следующей неделе — трёхмерные данные и множества, затем графы. А тексты, последовательности и рекуррентные сети подождут своего блока.